

EvoLUTe+: Fine-Grained Look-Up-Table-based RTL IP Redaction

Rui Guo*, M Sazadur Rahman**, Jingbo Zhou*, Hadi M Kamali**,
Fahim Rahman*, Farimah Farahmandi*, Mark Tehranipoor*

*Department of Electrical and Computer Engineering, University of Florida

**Department of Electrical and Computer Engineering, University of Central Florida

{guor, jingbozhou}@ufl.edu, {mohammad.rahman, hadi.mardanikamali}@ucf.edu,
{fahimrahman, farimah, tehranipoor}@ece.ufl.edu

Abstract—Hardware obfuscation is an active trustworthy design technique targeting threats in the IC supply chain, such as IP piracy and overproduction. Recent research on Intellectual Property (IP) protection technologies suggests that using embedded reconfigurable components (e.g., eFPGA redaction) could be a promising approach to hide the functional and structural information of security-critical designs. However, such techniques suffer from almost prohibitive overhead in terms of area, power, delay, and testability. This paper proposes an obfuscation technique called EvoLUTe+, which is a unique and more fine-grained redaction approach using smaller reconfigurable components (e.g., Look-Up Tables (LUTs)). EvoLUTe+ achieves fine-grained partitioning, sub-circuit coloring, and scoring of IP, and then encrypts the original IP through the substitution of some sub-circuits. Different attacks are used to test the robustness of EvoLUTe+, including structural and machine learning attacks, as well as Bounded Model Checking (BMC) attacks. The overhead of the obfuscation design is also analyzed. Experimental results demonstrate that EvoLUTe+ exhibits robustness with acceptable overhead while resisting such threat models.

Index Terms—Logic Locking, Re-configurability, eFPGA IP redaction, Universal Function, IP Protection.

I. INTRODUCTION

In today's IC industry, driven by rising manufacturing costs, the pressure of tight time-to-market schedules, and the urgent need for emerging technologies, numerous companies are embracing a globalized, horizontally integrated approach to structure their supply chains. [1]. This horizontal approach entails the collaboration of multiple autonomous entities, each bearing responsibility for distinct phases within the IC supply chain. In such a globally distributed supply chain, different companies or entities can focus on their respective areas of expertise, forming a synergistic collaboration. For example, some companies may focus on IC design and development, while others specialize in IC manufacturing and packaging, and there may be some companies dedicated to testing and validation, among other stages. The advantage of this horizontal mode is that each entity can achieve higher efficiency and specialization in its own domain, and through close cooperation, the entire supply chain can respond to market demands and technological changes more rapidly. At the same time, the horizontal mode can reduce risks for individual companies. Since different entities jointly undertake different segments of the supply chain, the failure of one company will not have a fatal impact

on the entire supply chain. Instead, other entities can continue to operate, maintaining the stability and continuity of the supply chain [2].

To protect integrated circuits (ICs), researchers have proposed numerous design solutions to secure the supply chain. These solutions aim to prevent security threats such as IP piracy, counterfeiting, and unauthorized access [3]. Watermarking, IC metrics, IC camouflage, and hardware obfuscation are common solutions [4]–[7]. Among these strategies, logic locking stands out as an effective method. It involves the insertion of supplementary key gates into the circuit to conceal the functionality of IP/IC. After the chip is manufactured, the key input is typically configured using on-chip tamper-resistant storage. Consequently, the design will only operate correctly when the accurate key input value is supplied [8]. However, with the continuous advancement of attack techniques, researchers have discovered certain limitations and vulnerabilities in traditional logic locking methods [9]–[12].

In recent years, a new approach to logic locking has gained momentum, known as the IP redaction method based on eFPGA [13]–[16]. eFPGA, as an embedded FPGA technology, enables SoC-level IP redaction. Users can choose to redact the entire IP asset [13] or its partial modules [14], [15] and implement the selected IP in the eFPGA embedded in the SoC. In Figure 1, an example is shown where Module 2 is replaced by a customized eFPGA structure. Modern FPGA specialization tools, such as OpenFPGA [17] and FABulous [18], enable designers to start from high-level Hardware Description Language (HDL) modules and generate corresponding soft eFPGA IP. These soft eFPGA IPs can be integrated and synthesized with the rest of the IP, forming a complete system. The bitstream programming of eFPGA serves as the key, and attackers must recover the complete bitstream to operate the entire system and eFPGA normally. Recent research has explored the parameters of eFPGA-based IP redaction, and the results show that coarse-grained eFPGA-based IP redaction presents as a hard SAT instance, which cannot be disrupted by any existing I/O query-based attacks [15]. In addition, as a regular general-purpose device, eFPGA can implement arbitrary functions. Its symmetrical structure and uniform distribution of CLB/LUT make structural attacks difficult to apply. Also, the

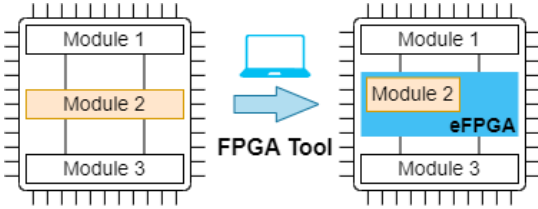


Fig. 1: eFPGA-based redaction replaces IP module(s) with a reprogrammable structure.

size of configuration bitstreams increases exponentially with the complexity of eFPGA architecture. This trend significantly expands the key space, enhancing the resilience against I/O query-based attacks [13], [14]. Although this eFPGA-based IP redaction method adds complexity and security to logic locking to some extent, it also brings new challenges. In particular, regarding the area overhead issue [13], [19], the eFPGA's IP redaction method is subject to many restrictions, resulting in a very narrow range of target IPs that designers can choose.

This paper proposes a novel fine-grained IP redaction method called EvoLUTe+, which replaces parts of IP using reconfigurable components such as Lookup Tables (LUTs), and utilizes the initial configuration of LUTs as an encryption key to address various forms of threats in logic locking. The core innovation of EvoLUTe+ lies in transforming complex IP designs into finely detailed node-level circuit diagrams, and it proposes a systematic strategy for circuit partitioning and reconstruction. Specifically, EvoLUTe+ deploys a proprietary advanced circuit partitioning tool based on conflict graph coloring, which can decompose the target IP into multiple sub-circuit units with adjustable granularity. Starting from the finest granularity, the subcircuits are gradually merged into coarser granularities through iterations, thus achieving a progressive transition of the circuit from the finest to the coarsest granularity. Additionally, it introduces inter-subcircuit relational modeling techniques, providing precise decision-making references for targeted redaction, thereby significantly enhancing the effectiveness and security of intellectual property protection while ensuring the functional integrity of the circuits.

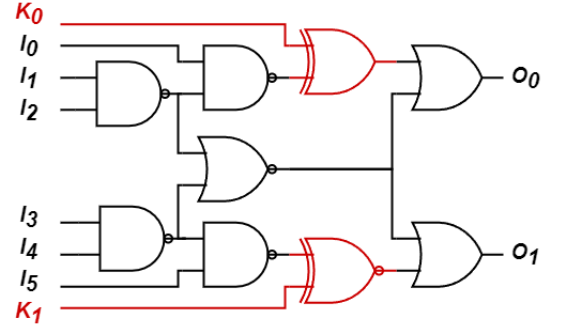
The rest of this paper is organized as follows: Section II presents the background on key-based obfuscation and eFPGA redaction. The proposed approach's detailed information is provided in Section III. Section IV demonstrates the effectiveness of EvoLUTe+ through experiments and performs security analysis. Finally, Section V concludes the paper.

II. BACKGROUND

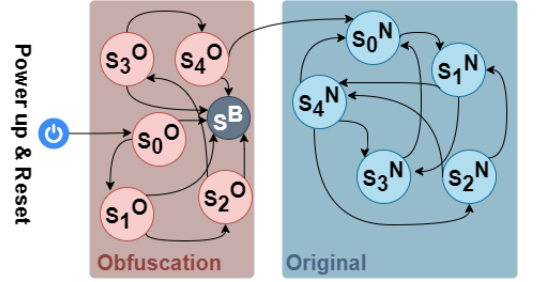
In this section, the preliminary research on logic locking and deobfuscation attacks is first introduced. Then, the existing programmable fabrics-based logic obfuscation and their limitations are provided as well. In the end, this section delves into potential threats to establish a comprehensive understanding of the problem background.

A. Logic Lockings and Deobfuscation Attacks

Explicit key and implicit key are two different approaches to logic locking used for chip protection. Figure 2 shows simple



a) Explicit key logic locking



b) Implicit key logic locking

Fig. 2: Example of logic locking. (a) Explicit key logic locking (key-based), and (b) implicit key logic locking (key less).

examples of logic locking. Explicit key logic locking involves inserting additional logic gates into the circuit and using a specific key to control the behavior of these gates [9]–[12], [20]–[23]. This key must be correctly input by authorized users or devices after chip fabrication to unlock and activate these gates, enabling the design's functionality. As shown in Figure 2(a), the implementation of explicit key logic locking is relatively straightforward, as both designers and authorized users are aware of the specific key to be used.

For the explicit key obfuscation, SAT-based attacks are a common approach in the deobfuscation field, utilizing SAT solvers to parse the Boolean logic within the logic locking design and deduce the unlocking key. The main steps of the SAT-based attack are illustrated in Figure 3 and described as follows. Firstly, construct a miter circuit containing two copies of the locking netlist and one oracle with the same primary inputs but different key inputs. The purpose of this design is to find a set of primary inputs that, when two different keys are applied simultaneously, produce different outputs for the two netlist copies, known as Distinguishable Input Pairs (DIPs). Next, the attacker runs the unlocked chip (oracle) using the previously found DIPs and collects the correct responses that can only be generated with the correct key. Then, the attacker adds all the collected DIPs and their corresponding correct responses as constraints to the SAT circuit. The SAT solver attempts to satisfy these constraints to find the key that meets the conditions. The entire process iterates continuously as the attacker seeks more DIPs and corresponding correct responses,

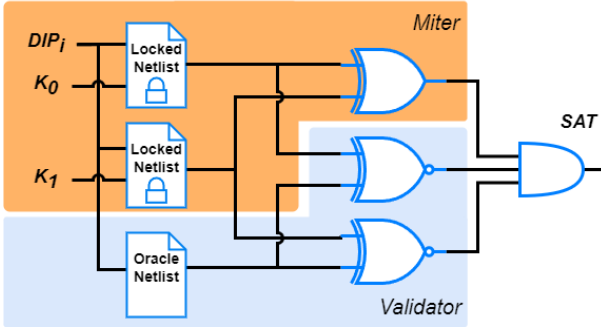


Fig. 3: Concepts of miter and validation in combinational SAT attacks.

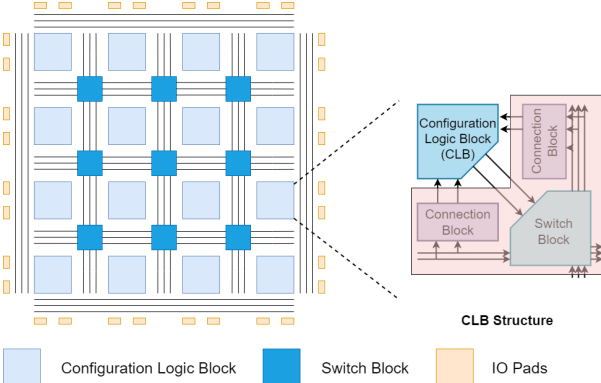


Fig. 4: FPGA Architecture. It contains configurable logic blocks, including LUTs, flip-flops, and routing logic for a MUX-based crossbar

adding them as constraints to the SAT circuit. This way, the SAT solver gradually approaches the correct unlocking key. Finally, when the attacker cannot find more DIPs, they call the SAT solver again using all the previously collected DIPs, which eventually reveals the correct unlocking key.

Implicit key logic locking, on the other hand, is often used for the protection of sequential circuits, targeting the State Transition Graph (STG) of the circuit [24]–[29]. Only when a specific input sequence is applied to trigger a specific state, the design’s functionality can be unlocked, as shown in Figure 2(b). This particular input sequence is referred to as the “implicit key” and it is not directly exposed to designers or users during the manufacturing and design processes.

In the deobfuscation process of sequential circuits, special attention must be paid to the circuit’s state information. Specifically, to isolate the original state feedback paths, it is necessary to truncate the key flip-flops (FF), making their input and output ports the primary inputs and outputs (PI/PO). Then, the original sequential circuit design is duplicated multiple times and connected in a cascade to form a circuit structure that covers several complete clock cycles. In this expanded circuit, an SAT solver can be used to search for a Distinguishing Input Sequence (DIS) that differentiates between the original design and the replicated design.

B. Programmable Fabrics Redaction-based Obfuscation

eFPGA redaction technology has received extensive attention in recent years [13], [14], [16]. As a regular general-purpose device, eFPGA can realize any function

through it. It has been shown that coarse-grained eFPGA-based IP redaction renders as a hard SAT instance that cannot be broken by any existing I/O query-based attacks [3]. This technique allows designers to implement a portion of the design into a programmable fabric while the remainder remains in the traditional ASIC design and manufacturing flow. Figure 4 shows the structure of an eFPGA, which contains configurable logic blocks (CLBs), including look-up tables (LUTs), flip-flops, and routing logic for a MUX-based crossbar. These logic blocks can be programmed to perform various functions. For configurable routing, the FPGA also contains MUX-based switch boxes and connection blocks. This reconfigurability allows designers to implement critical parts of the design after fabrication by configuring the bitstream, which is the key.

Due to the reconfigurability of eFPGA, the entire design can be fabricated by any untrusted offshore foundry without programming the eFPGA. This feature adds to the confusion of the overall design and enhances security. After finalizing the design, a trusted authority proceeds to program the eFPGA, thereby restoring the full functionality of the entire design, ensuring that the design will function properly after being activated.

To launch an SAT attack, attackers first need to design an SAT solver and manipulate circuit inputs to treat the configuration bitstream as a “key”. When there are combinational loops, the SAT solver fails, producing unstable results or repeatedly returning different input patterns, which originate from the reconfigurable routing network of eFPGAs. The reconfigurable logic sequences represented by LUT/CLB chains interconnected by the reconfigurable network increase the polynomial complexity of the SAT formula [14].

In [16], the authors adopted a method of redaction from high-level (C-based) designs to maximize security metrics until the area overhead threshold is reached. On the other hand, in [13], eFPGA structures were generated by selecting various modules of the design. During IP redaction, designers need to balance security and other design factors, such as area and timing, and choose the appropriate redacted modules suitable for the eFPGA structure.

C. Limitations of eFPGA Redaction-Based Obfuscation

Although programmable fabric-based redaction techniques provide a certain level of security for designs, they also bring several challenges. Some of these challenges include customizing/selecting fabric configurations, deciding which IP blocks should be implemented on the programmable fabric, and dealing with the integration and implementation overhead involved. In certain cases, these challenges can lead to significant area overhead [30], making these methods unusable [31], [32]. However, due to limited resources and specific fabric size constraints, designers cannot arbitrarily choose the target modules. After determining the redacted modules, it is also necessary to ensure compliance with the fabric size requirements. However, the minimum fabric size is influenced by multiple factors. As an example, the module’s interface, including the number of inputs and outputs, directly influences

the necessary allocation of I/O blocks. Similarly, the count of state elements, such as registers and flip-flops, affects the configuration of logic blocks within the design. These factors lead to variations in the final size of the eFPGA, resulting in suboptimal internal resource utilization for the structure used for redaction.

D. Threat Model

In key-based logic locking attacks, the attacker's goal is to recover the key used to obfuscate the IP, thus restoring its original functionality. If the adversary successfully recovers the key and retrieves the locked gate-level netlist from the locked GDSII or activated chip through reverse engineering, the key-based reverse engineering attack is considered successful.

To evaluate the robustness of our proposed method, we employ the conventional oracle-guided threat model for evaluation. [33]. In this threat model:

- (I) The attacker can be any individual or entity involved in the IC lifecycle, including untrusted foundries and malicious end-users. The attacker may have access to the locked GDSII (in offshore foundries) and activated chips. In both cases, the attacker can retrieve the locked gate-level netlist through reverse engineering techniques [3].
- (II) The attacker can access activated/unlocked chips, thus having the ability to apply various attack methods. For instance, I/O-based attacks like SAT attack [33] can be applicable in this situation. Additionally, the attacker may employ machine-learning-based attacks [34], to further attempt key recovery.
- (III) The attacker operates with minimal constraints on information access, except for possessing the correct unlock key. Moreover, the attacker may have the capability to access scan chains, which can be exploited to partition sequential circuits into smaller sub-circuits [35], allowing for independent attacks on each sub-circuit.

III. PROPOSED SCHEME: EvoLUTE+

Building a robust fine-grained programmable fabrics-based redaction obfuscation to withstand attacks requires meeting the following key requirements:

- (I) Adaptability: The obfuscation scheme should have high adaptability, meaning it should be applicable to different scales and types of IP designs. It should be flexible enough to be applied to various circuits and functional modules.
- (II) Lightweight: Both the obfuscation process and its outcomes should be lightweight to minimize the impact on the design. The obfuscated design should incur minimal area overhead, enabling easy deployment in practical applications.
- (III) Security: The obfuscation scheme must provide sufficient security to withstand existing mainstream attack methods, such as SAT-based attacks and model-checker-based attacks.

A. EvoLUTE+ Overview

Our proposed EvoLUTE+ is an obfuscation scheme applicable to RTL sequential designs, capable of meeting the key requirements mentioned above. The main objective of EvoLUTE+ is to achieve fine-grained redaction of sequential

designs while preserving SAT resilience, allowing us to selectively replace multiple sub-circuits by LUTs and keeping the area overhead within acceptable level.

The workflow of EvoLUTE+ is shown in Figure 5. The following sections discuss in detail the key components of EvoLUTE+ and how they work, starting with the original RTL design of the IP asset. First, the original RTL design needs to be compiled into a gate-level netlist using Synopsys Design Compiler (*Preprocessing*). Then a Node-based circuit graph is constructed based on the gate-level netlist, and all Nodes in the graph are labeled according to whether they are sequential components. A labeled circuit graph makes it easy to isolate components such as flip-flops from combinational logic. Subsequent steps will only process combinational logic elements, while labeled as sequential logic elements such as flip-flop will not be affected by circuit splitter or coloring and scoring tool (Step 3: *Sequential Isolation*). In step 4, we use the developed graph-based circuit splitter to split the original IP into multiple sub-circuits and keep the mutual connection relationship unchanged. The sub-circuit decomposition step is independent of the modules in the design, and the final decomposition result may be only a small part of the module, with the minimum granularity including only one gate. At the same time, a top-level RTL is created in which all sub-circuits are instantiated to reconstruct the entire IP functionality (Step 5: *IP reconstruction*). To select sub-circuits for replacement, EvoLUTE+ performs conflict graph coloring to identify conflicting sub-circuits. This approach ensures that an attacker encounters each sub-circuit individually and cannot gain any leverage over another by compromising one sub-circuit. EvoLUTE+ then assigns each conflicting sub-circuit a score based on how well it interacts with other sub-circuits. The higher the score, the more interactions with adjacent sub-circuits (step 6: *Sub-circuit coloring & scoring*). After determining the target sub-circuits based on the results of the previous step, use the internally developed Python tool to generate MUX-based circuits (step 7: *LUT generation*), and then replace these target sub-circuits (Step 8: *Redaction*). And the configuration of the MUX tree-based circuit will be the key. Afterward, the top-level RTL generated by the circuit splitter tool is used to integrate all sub-circuits and LUTs to implement the entire IP. Then proceed to the regular ASIC design process. Once a single IC is packaged, the previously generated bitstream is used to unlock LUT-based redaction IP capabilities. From *Sequential Isolation* to *Redaction*, there will be a gradual iteration to achieve splitting from the finest granularity (a sub-circuit only contains one gate) to the coarsest granularity (sub-circuits cannot be merged into coarser granularity).

With this fine-grained partitioning and reconstruction approach, EvoLUTE+ demonstrates excellent adaptability to different scales and types of IP designs. It can be applied to various circuits and functional modules, enabling designers to selectively protect critical parts of IP according to their needs. Moreover, EvoLUTE+ offers high flexibility and ease of

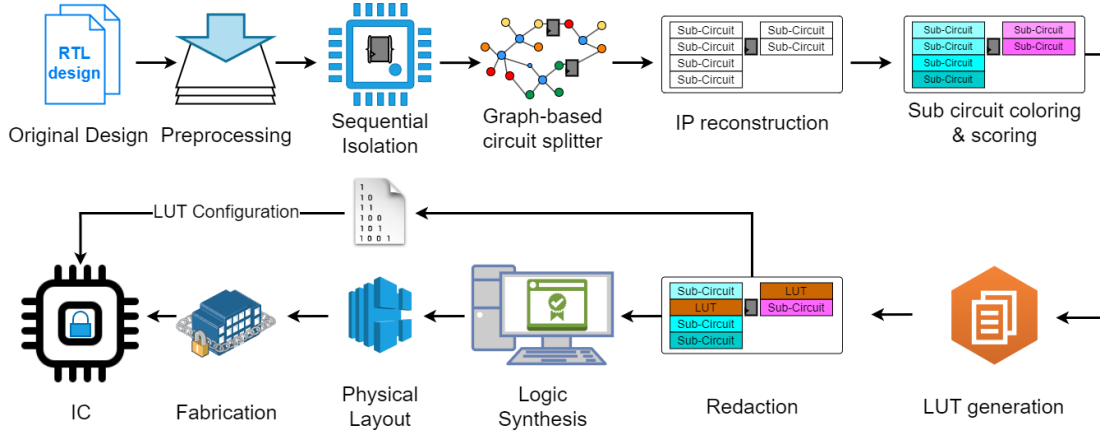


Fig. 5: IC design flow of EvoLUTe+

use in the design, seamlessly integrating into existing design flows. Additionally, our approach is optimized specifically for protecting sequential circuits, considering the complexity and wide application of sequential logic.

B. Sequential Isolation

In EvoLUTe+, we first label and isolate sequential logic elements such as flip-flops in the design. This step is crucial because flip-flops play a significant role in sequential circuits, serving as critical elements for storing and controlling signal propagation. Figure 6 illustrates a schematic diagram of the isolated design, where the gray parts are labeled flip-flops, and the combinational logic is connected to each flip-flop. During this process, flip-flops often have single-signal inputs and are connected to multiple logic cones as outputs. These isolated flip-flops remain unchanged in the subsequent partitioning and reconstruction process, preserving the important sequential information associated with them, ensuring that the obfuscated design retains consistent performance and sequential behavior with the original design.

In the preprocessing step shown in Figure 5, the first task is to convert the RTL design into a gate-level netlist using Synopsys Design Compiler (DC). Once the gate-level netlist is obtained, the next step is to construct the Node Graph based on the generated netlist. The Node Graph is a crucial data structure that represents the design as a graph, where each node corresponds to a specific logical gate and also records the connections between nodes.

After generating the Node Graph, the collection of sub-circuits is initialized, with each sub-circuit initially containing only one gate. Then, the specific nodes in the graph are scanned, and their corresponding sub-circuits are labeled. This includes different types of flip-flops, clocks, inputs, and outputs. Flip-flops are crucial components in sequential circuits, used for storing and stabilizing signal states. By labeling flip-flop nodes, we can ensure the protection of the sequential logic in the design during the subsequent redaction process. In addition to flip-flop nodes, it is also necessary to label clock nodes, input/output nodes, and others. These labels help maintain the correct functionality of the design during the

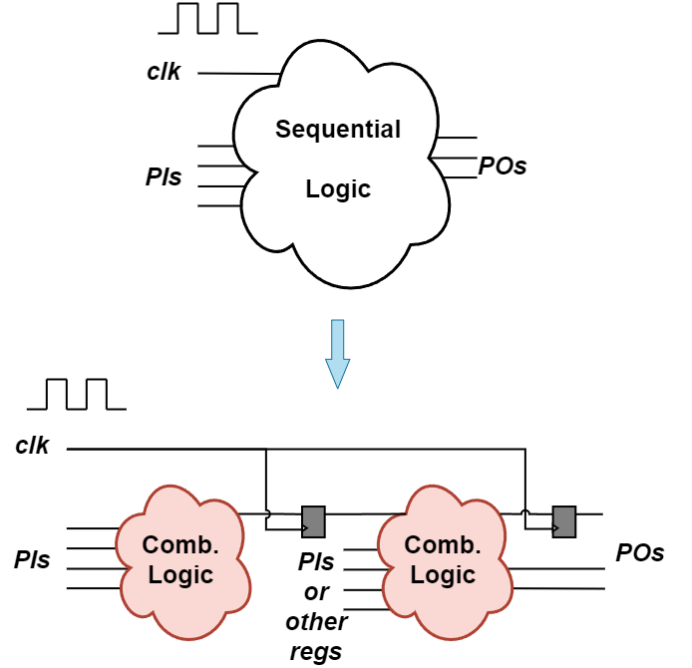


Fig. 6: Sequential and combinational logic isolation.

redaction process. The block diagram of the preprocessing and isolation is shown in Figure 7.

C. Graph-Based Circuit Splitter

In EvoLUTe+, once the initialization steps are completed and special nodes are labeled, the Graph-Based Logic Splitter starts iteratively merging the sub-circuits and their neighboring sub-circuits. This highly efficient merging process greatly enhances the scalability of the EvoLUTe+ framework. In each iteration, the Splitter prioritizes merging subcircuits containing fewer nodes, and each subcircuit is merged at most once per iteration. This is to ensure that the size differences between subcircuits within the same iteration are not too large. To ensure the correctness of the circuit partitioner, EvoLUTe+ also performs functional verifications. This ensures that the splitting and merging operations do not introduce design errors or affect functionality. Specifically, after each sub-circuit is generated, the top-level design is reconstructed and all sub-circuits are

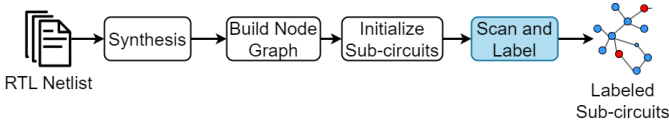


Fig. 7: Block diagram of preprocessing and isolation of RTL IPs. After the Node graph is established, all components are labeled, and sequential circuit components such as flip-flop will be isolated separately and will not undergo subsequent processing such as splitting.

Algorithm 1 Circuit Split Algorithm

```

1: procedure GNENRATE_SUB_CIRCUITS
2: Input:  $\{M_{cg}, config\}$ 
3: Output:  $TOP.v, M_{cg}$ 
4:  $InitializeCategory(M_{cg})$ 
5: for  $sub\_ckt$  in  $M_{cg}$ :
6:   if  $(sub\_ckt.processed \neq True) \&$ 
7:      $(sub\_ckt.label == normal)$  then
8:      $ConnetedCkts = get\_connected\_ckts(sub\_ckt)$ 
9:      $minCkt = get\_min\_estimist\_key\_len(ConnetedCkts)$ 
10:     $merge(sub\_ckt, minCkt)$ 
11:     $sub\_ckt.processed = True$ 
12:     $update\_boundary(sub\_ckt)$ 
13:     $generate\_sub\_ckt\_verilog(sub\_ckt)$ 
14:  $TOP.v = update\_sub\_ckts(M_{cg})$ 
15: return  $TOP.v, M_{cg}$ 

```

instantiated. We use Synopsys VCS to verify the waveforms of multiple clock cycles of the original design and the reconstructed design when given the same input stimulus, and the reconstructed design was considered functionally equivalent to the original design when their outputs were all the same. Alg. 1 is the pseudocode of the splitting algorithm. The algorithm starts iterating from the finest granularity initial state, treating each gate circuit as an independent sub-circuit. In each subsequent iteration, according to Alg. 1, adjacent sub-circuits are gradually merged, increasing the size of the sub-circuits until reaching the final state ($iter_N$) where no further merging is possible.

In Alg. 1, Splitter takes the initialized and labeled circuit model (M_{cg}) and the configuration parameters provided by the designer ($config$) as input. $Config$ specifies constraints such as the maximum allowable size of sub-circuits and key length limits. The core of the algorithm is an iterative process where, in each iteration, Splitter loops through all sub-circuits. For each sub-circuit that is not yet merged and does not belong to special isolation nodes, Splitter retrieves all unmerged neighboring sub-circuits connected to it (Lines 6-8). In order to prevent the size of the sub-circuit from being too large during one iteration, we limit each sub-circuit to being merged at most once in one iteration. For each candidate neighboring sub-circuit, the Splitter first simulates the merge operation to estimate the number of I/O ports of the merged sub-circuit and calculates the potential key length introduced based on the number of I/O ports to determine if it exceeds the designer-specified range (Line 9). Next, among the candidates meet the merging conditions, the algorithm selects the one with the minimum estimated key length introduced and performs the actual merge operation with the current sub-circuit (Line 10). After the merge, Splitter identifies the boundaries of the new sub-circuit, redefines the I/O ports, and generates

the corresponding RTL netlist (Line 12-13). Through these iterative steps, Splitter gradually merges sub-circuits, ensuring that each sub-circuit is merged at most once in the same iteration, and generates a collection of sub-circuits of different sizes. Thus, designers can flexibly choose between fine-grained or coarse-grained sub-circuit views based on actual needs to achieve the desired circuit implementation.

We illustrate the process of merging from fine-grained to coarse-grained using the b02 from ITC'99 [36] as an example, as shown in Figure 8. The figure displays all nodes of the benchmark circuit, where nodes of the same sub-circuit share the same color. After the initialization step, we first marked all DFFs (flip-flops) and ensured that they were not subject to merging in subsequent iterations. These labeled flip-flops are represented as relatively larger nodes in Figure 8 (U_REG, STATO_REG_0_, STATO_REG_1_, STATO_REG_2_). By keeping these FFs unchanged, we ensured the correctness and stability of the sequential logic in the original circuit. Next, in each iteration, the splitter prioritizes the merging of neighboring sub-circuits to form larger sub-circuits. In Figure 8, we can observe that in $iter_0$, many fine-grained sub-circuits are retained (14 sub-circuits that can be merged and 4 DFF sub-circuits), with each sub-circuit containing a small amount of logic and gate-level netlist. As the iterations progress, we gradually merge some adjacent sub-circuits, as shown in $iter_1$, $iter_2$, and $iter_3$. From $iter_0$ to $iter_1$, the original 18 sub-circuits are merged into 12. In addition to 4 DFF sub-circuits, there are 8 combinational logic sub-circuits. From $iter_1$ to $iter_2$, the remaining 8 combinational logic sub-circuits are merged into 4, and the original 4 DFF sub-circuits remain unchanged. By $iter_3$, EvoLUTe+ has merged all sub-circuits, leaving only 3 combinational logic sub-circuits and 4 DFF sub-circuits. These remaining sub-circuits are relatively large in size compared to the finer granularity of the initial iterations. With the merging of sub-circuits, the overall circuit complexity and scale decrease, and some smaller sub-circuits are combined into larger functional blocks. For ease of observation, we assign different colors to different sub-circuits in Figure 8, it is important to note that here is not the same concept as the Sub-Circuits Coloring discussed in Section III-D.

D. Sub-Circuits Coloring and Scoring

In EvoLUTe+, numerous sub-circuits that could be redacted are generated in each iteration. Design engineers need to strike a balance between security and cost and may not redact all the subcircuits that meet the criteria.

Therefore, design engineers need to adopt a selective strategy, choosing only a limited number of sub-circuits for redaction in each iteration. This approach allows them to minimize the introduced overhead while maintaining security. Designers can select sub-circuits that have a greater impact on the overall circuit performance but can effectively enhance security based on practical needs and resource constraints. The selection strategy we provide here can also be used when the designer has no preference for the selection. Once the



Fig. 8: Circuit splitting for b02 [36] in EvoLUTe+.

circuit is splitted, we utilize graph coloring techniques [37] to implement a unique strategy called Redaction-oriented Conflict Graph Coloring (RoCGC). We define two directly connected subcircuits as conflicting circuits, the distance between them is 1. We use RoCGC to color the sub-circuits, aiming to provide design engineers with a set of references to select appropriate sub-circuits during the redaction process. Specifically, EvoLUTe+ avoids simultaneously selecting two sub-circuits that are directly connected, as doing so may lead to potential information leakage, enabling malicious attackers to obtain information from one sub-circuit to another. This coloring strategy ensures that the selected sub-circuits maintain independence and security to the maximum extent possible when interconnected and interacting with each other.

In Figure 9, we show the conflicting circuit of the reference circuit b02 at $iter_0$. We define circuits that are not directly connected as conflicting circuits. Figure 9 (a) shows the sub-circuits recommended to be redacted at $iter_0$. For the convenience of observation, we hide other sub-circuits in the figure. It can be seen that these recommended circuits do not have any connection relationship with each other, that is, the distance between them is greater than 1, thereby avoiding potential information leakage caused by selecting two directly connected sub-circuits at the same time. Figure 9 (b) takes sub_ckt_4 as an example, we can see that sub_ckt_11 and sub_ckt_3 are directly connected to sub_ckt_4 , respectively marked by dashed circles. Neither sub_ckt_11 nor sub_ckt_3 belong to conflict circuits and will not be recommended for redaction. This result is consistent with the behavior of our algorithm.

In EvoLUTe+, once the conflict circuits are identified, the next crucial step is to score each conflict sub-circuit. Although RoCGC can effectively identify recommended conflicting circuits, for large designs, the number of sub-circuits available for designers to choose from is still very large. The purpose

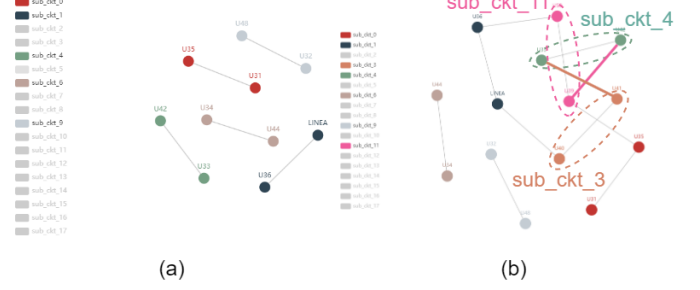


Fig. 9: Example of conflicting sub-circuits for b02 [36] at $iter_0$. (a) Conflicting sub-circuits are not connected to each other (sub_ckt_0 , sub_ckt_1 , sub_ckt_4 , sub_ckt_6 , and sub_ckt_9), and (b) sub_ckt_3 and sub_ckt_11 are directly connected to sub_ckt_4 , so they are not recommended for redaction.

of scoring is to provide designers with a reference for selecting some sub-circuits for redaction.

The scoring process mainly assesses the impact of the conflict sub-circuits from two aspects:

Sub-circuit Connectivity: The evaluation of the connections between the conflict sub-circuit and other sub-circuits. If the conflicting sub-circuit is connected to a larger number of other sub-circuits, its impact on the overall structure and function of the IP may be more significant, as redaction may affect multiple neighboring sub-circuits. During the scoring process, EvoLUTe+ counts the number of direct connections between the conflict sub-circuit and other sub-circuits and quantifies its impact based on the number of connections.

Node Connectivity: The assessment of the strength of connections between the conflict sub-circuit and other sub-circuits. Specifically, EvoLUTe+ considers the number of nodes connecting the conflict sub-circuit to others. While considering the connectivity of sub-circuits to each other, extreme cases should also be considered. For example, although one sub-circuit is connected to multiple other sub-circuits, they are small in scale and only connected to each other through one gate/node. Another situation is when a sub-circuit contains many gates, with more than half of the gates/nodes connected to each other, although it is connected to only a few other sub-circuits. In this case, the impact of this sub-circuit on the overall structure will be much greater. As sub-circuits are merged, the circuit scale is not exactly the same. Therefore the scoring process in EvoLUTe+ takes into account both the number of connecting nodes and their importance to quantify the node connectivity between the conflict sub-circuit and other sub-circuits.

By considering the above factors comprehensively, each conflict sub-circuit receives a score. Then, EvoLUTe+ ranks the conflict sub-circuits from high to low based on their scores. The scoring model we propose is as follows:

$$score_{k|j} = nodes_{total} \times w_{ckt} \frac{subckt_{connected}}{n} + \frac{subckt_{connected}}{n} (1 - w_{ckt}) \sum_{i=0}^n \frac{nodes_{connected}}{nodes_{ckt_i}} \quad (1)$$

In Eq.1, w_{ckt} represents the weight of the circuit. The range of w_{ckt} is [0,1], which is given by the designer. The higher

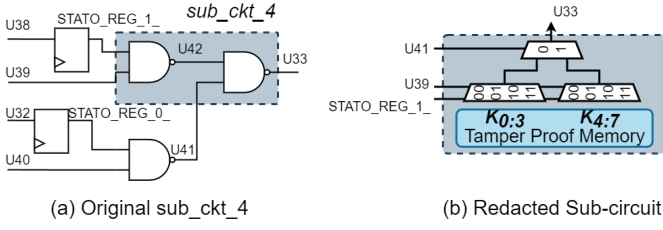


Fig. 10: Map *sub_circuit_4* to MUX-based LUT.

w_{ckt} , the more emphasis is placed on sub-circuit connectivity. Conversely, if the value of w_{ckt} is lower, node connectivity is emphasized more. In $iter_j$, there are n sub-circuits in total, and for each sub-circuit k , there are $subckt_{connected}$ sub-circuits connected to it. At the same time, for each sub-circuit i connected to sub-circuit k , there are $nodes_{connected}$ nodes in sub-circuit i connected to sub-circuit k . When sub-circuit i does not have any connection to sub-circuit k , $nodes_{connected}$ will be 0. This unique definition configured by the coefficient w_{ckt} enables designers to select sub-circuits based on both node-level and circuit-level attributes.

E. Granular Redaction

In Figure 10 (a) and Figure 10 (b), we still take the *sub_circuit_4* at $iter_0$ of the b02 as an example to demonstrate how to map logic sub-circuits with 3 inputs and 1 output to LUTs. *sub_circuit_4* has 3 inputs, namely *STATO_REG_1_*, *U39*, and *U41*, and the output is *U33*, as shown in Figure 10 (a). To replace this sub-circuit, we use a LUT with 8 configuration bits (ranging from 0 to 7). These configurations are treated as keys stored in tamper-proof memory, similar to the eFPGA-based redaction approach. By configuring LUTs with different values, various logic functions can be implemented to achieve circuit obfuscation and protection.

IV. EXPERIMENTAL RESULTS AND SECURITY ANALYSIS

In this section, we evaluated EvoLUTe+ and validated its performance and industrial scalability using different circuit benchmarks. We selected several commonly used open-source designs from the industry as our test cases and evaluated the security performance of EvoLUTe+ by conducting BMC or SAT attacks. These test cases were chosen to represent circuits of varying complexities and scales, ensuring a comprehensive and accurate assessment of the security of EvoLUTe+. To conduct the security evaluation, we varied the number of redacted sub-circuits to observe the success rate and time complexity of the attacks, thus verifying EvoLUTe+'s resilience to attacks. Additionally, we analyzed the area overhead of EvoLUTe+ and studied its relationship with the number of redacted sub-circuits. This helps us understand the scalability of EvoLUTe+ in industrial applications and whether it is suitable for large-scale complex circuit designs.

A. Experimental Setup

In order to thoroughly understand and evaluate the performance of EvoLUTe+, we applied our proposed obfuscation method to benchmark circuits such as ITC'99 [36], EPFL [38],

TABLE I: Benchmark Circuits Specification

Circuit	Area (μm^2)	#Inputs	#Outputs	#FF/LAT
Sine	6465.5	24	25	0
Square	25439	64	128	0
Multiplier	34586	128	128	0
Arbiter	5759	256	129	0
Log2	42220	32	332	0
pcode	2175.1	10	1	163
FIR_filter	11885.5	34	32	443
IIR_filter	11549.3	34	32	662
md5	10295.6	516	131	269
b15	10417.3	38	70	449
b22	21636.3	34	22	735

and CEP [39]. In these experiments, we considered various specifications of each benchmark circuit, including the number of gates, input/output ports, flip-flops, and the overall circuit area. We intentionally selected benchmark circuits of different scales and ports to assess EvoLUTe+'s performance under various scenarios. The specifications of these benchmark circuits are provided in Table I. For each test benchmark, we followed the scoring strategy proposed in Section III-D and conducted the redaction operations in the recommended order of sub-circuits. This was done to thoroughly investigate the security effects of EvoLUTe+ in various scenarios and to obfuscate while maintaining acceptable implementation overhead. All of our SAT attacks were conducted in an environment equipped with a 32-core 2.6 GHz CPU and 64 GB of memory, while BMC attacks were carried out in an environment with a 64-core 2 GHz CPU and 512 GB of memory. To conduct the experiments, we utilized a set of tools, including Synopsys Design Compiler, Cadence JasperGold, nuXMV [40], ABC [41], and Python 3.9.

B. SAT Attack Results

For the benchmark circuits listed in Table I, we validated the security of the combinational logic through SAT attacks. We selected the results from $iter_1$ and $iter_2$ of the circuit partitioning tool to assess the impact of redaction granularity on SAT resilience and area overhead. As elaborated in Section III-C, the sub-circuits generated in the $iter_1$ phase exhibit a finer granularity, implying that they are divided into smaller units and their total count is more substantial. For various benchmark circuits and the outcomes of each iteration ($iter_1$ and $iter_2$), we conducted replacements of selected numbers of sub-circuits with a corresponding count of LUTs (as indicated in the first column of Table II). Subsequently, we implement SAT attack [33], with a timeout set at 24 hours. The various rows in Table II represent the redaction scenarios for different numbers of sub-circuits, and we extensively explored these redactions. Within each row, we present the corresponding area overhead (a%), key size ($||k||$), and SAT attack execution time (t (h)), with these details displayed in different columns. We incrementally increased the number of sub-circuits until the predetermined timeout was reached. We implemented redactions of the same scale in the eFPGA to assess based on coarse-grained redaction. During this phase, we reported the associated area overhead, bitstream size, and SAT attack execution time.

From the experimental results, it is evident that the number of redacted sub-circuits is a key variable in our study, holding significant influence for each benchmark circuit. Our observations indicate that as the count of redacted sub-circuits increases, we observe a gradual rise in area overhead, key size, and SAT attack time. Meanwhile, concerning each benchmark, we noticed that in the $iter_2$, due to the relatively larger size of sub-circuits, fewer sub-circuits are required to meet the timeout compared to $iter_1$. However, the achievement of this effect comes at the cost of increased area overhead. Therefore, by employing a finer-grained redaction strategy, superior SAT resilience can be achieved, meaning a smaller area overhead can be attained while satisfying the time constraint. However, redaction based on eFPGA represents the coarsest form within this spectrum, thus requiring fewer subcircuits to meet the requirements of SAT resilience at the expense of an unacceptable area overhead.

Deciding when to stop iteration is a crucial issue. Typically, the decision to stop iteration is influenced by constraints set by the designer and performance requirements. These constraints include maximum subcircuit size, maximum key length, or other technical specifications related to circuit design. When iteration reaches the constraints set by the designer, iteration will be stopped at $iter_n$. This means that the size of subcircuits cannot be further increased or subcircuits further merged, so evaluation and selection need to be based on the results from $iter_0$ to $iter_n$. When deciding which iteration result to adopt, designers need to consider multiple factors, including overhead, security, and other performance metrics. Although we provide scores for each sub-circuit in each iteration as a reference for selection, we cannot guarantee to meet all designers' requirements because designers may have different emphases when designing circuits, which may vary due to project requirements, security requirements, or performance requirements. Therefore, the final decision needs to be based on a comprehensive evaluation and balance of specific circumstances and design objectives. In our experiments, we assume that designers do not have specific design styles or requirements, aiming to evaluate the typical performance of EvoLUTe+ under general conditions.

C. BMC Attack Results

In security analysis, BMC attacks are also a focus for assessing security. By performing bounded model checking on the circuit, attackers can search for possible unlock sequences or paths within a finite state space to bypass obfuscation. This makes BMC attacks an effective method. We verify security by launching BMC attacks on sequential logic. The open-source toolset RANE [42] provides a unified framework with a unique interface that leverages the functionality of commercial formal verification tools such as Cadence JasperGold and Synopsys Formality (e.g., scalability) to perform de-obfuscation at different stages.

Figure 11 provides the resilience assessment results of EvoLUTe+ against Oracle-guided BMC attacks on the sequential design benchmarks listed in Table I. We utilized Cadence

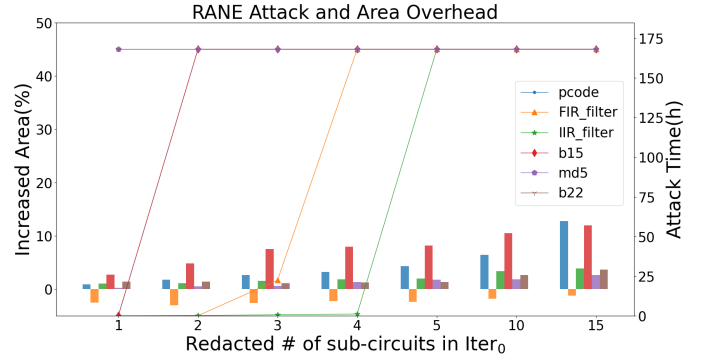


Fig. 11: RANE attack time (TO: 7 days) and area overhead. The bars represent the area increment compared to the original design, and the lines represent RANE attack execution time.

JasperGold as the model-checking engine for BMC attacks [42]. All attacks were evaluated within a 168-hour (7 days) timeout. In these experiments, we particularly focused on the robustness and security exhibited by EvoLUTe+ when facing Oracle-guided attacks.

Similar to the SAT attacks, we adopted a similar approach to gradually increase the number of redacted sub-circuits to evaluate the impact of sub-circuit quantity on BMC attack resilience. The objective of this experimental design was to comprehensively understand EvoLUTe+'s resistance to BMC attacks when facing different scales of redaction. By progressively introducing more LUTs, we could delve into how the attacker's results change when confronted with increasingly complex and refined obfuscated circuits. It is important to note that our experiments were conducted only under the finest granularity of redaction conditions, which will help reveal EvoLUTe+'s performance under the singular variable of sub-circuit quantity consideration.

In the case of BMC attacks, compromising EvoLUTe+ implies that an attacker can generate a set of DIPes that exposes the key. According to our experimental results, despite the successful disruption of obfuscation mechanisms through replacing a small number of sub-circuits, as the circuit size and complexity increase, even with the replacement of only a few sub-circuits, such attacks may potentially fail. This indicates that EvoLUTe+ exhibits stronger resilience against BMC attacks in scenarios involving larger scales and more complex circuits. Compared to combinational logic, obfuscating sequential circuits requires a lower number of sub-circuits. This can be attributed to the fact that sequential circuits need to consider multiple aspects such as timing constraints, clock signals, state transitions, and other factors. These inherent features make the process of BMC attacks more intricate. Sequential circuits must adhere to strict timing rules to ensure signal synchronization and state transitions. The presence of state machines and complex signal pathways further amplifies the complexity of attacks.

D. Machine-Learning-Based Attack Results

Oracle-less ML attack is a technique used for machine learning attacks that do not require access to a real "Oracle". This type of attack typically relies on black-box or zero-knowledge

TABLE II: SAT Attack and Overhead (Area) Comparison between LUT-based and eFPGA-based Redaction.

Redact	Iter ₁ (more fine-grained)															Iter ₂ (more coarse-grained)														
	Sine			Square			Log2			Multiplier			Arbiter			Sine			Square			Log2			Multiplier			Arbiter		
	a%	k	t(h)	a%	k	t (h)	a%	k	t(h)	a%	k	t (h)	a%	k	t(h)	a%	k	t (h)	a%	k	t(h)	a%	k	t (h)	a%	k	t(h)	a%	k	t (h)
5 LUTs	-14.5	24	0.41	-21	96	2.07	-13	28	0.59	-25.8	54	0.44	17	84	0.01	-18	304	1	-18.06	504	3.54	-9.85	68	1	31	17K	1	33	1.5K	0.9
25 LUTs	-3.22	356	2.76	-8	236	4.52	-5.4	184	2.54	-17.6	130	1.90	28	378	0.05	14	3K	2	-15.51	1.1K	6.25	-8.18	254	4.21	39	18.9K	1.9	41	5.9K	15.74
30 LUTs	1	648	4.05	-1	296	19.5	-4.7	292	3.32	-17.4	316	2.76	40	674	0.31	71	6.5K	5.1	-14	1.4K	TO	-2	312	6.5	42	21K	2.5	75	7.2K	TO
35 LUTs	9	736	5.83	-18.3	440	TO	-4	356	4.05	-17.1	384	3.08	47	738	0.41	152	12K	TO	-	-	-	1	424	7.26	47	27K	4.39	-	-	-
60 LUTs	17	1K	15.3	-	-	-	1	648	6.94	-15.8	640	5.62	53	1K	1.38	-	-	-	-	-	-	9	918	13.5	83	43K	13.9	-	-	-
90 LUTs	20	1.1K	TO	-	-	-	7	812	23.2	-12.4	1.2K	22.7	57	2.1K	4.01	-	-	-	-	-	-	17	1.1K	TO	94	46K	TO	-	-	-
150 LUTs	-	-	-	-	-	-	18	1.3K	TO	-12	13K	TO	71	3.2K	21.9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
1 eFPGA	354	9K	TO	201	14K	TO	129	16K	TO	151	17K	TO	311	11K	TO	323	7.5K	TO	189	11.7K	TO	105	14.6K	TO	138	16.3K	TO	258	10.2K	TO

methods, where the attacker cannot directly access internal information or obtain real data labels beforehand. However, by using machine learning algorithms and other techniques, they can conduct attacks externally to the model and attempt to infer sensitive information or achieve attack objectives. For example, the SWEEP attack [34], which depends on re-synthesis methods relying on re-synthesizing additional critical gates and learning optimization algorithms of EDA tools. Thus, the attacker only needs to access the locked gate-level netlist and apply the key values, then perform re-synthesis. With knowledge of the optimization algorithms used in re-synthesis, the SWEEP attack attempts to recover the original gate-level netlist state before locking, to reveal the protected logic design. We evaluated the performance of EvoLUTe+ using a machine learning-based SWEEP attack [34]. However, the machine learning-based oracle-less attack doesn't extract any keys from the obfuscated circuits of EvoLUTe+. This aligns with our expectations. During the process of transforming, mapping, and optimizing from RTL to gate-level design, the obfuscated circuit becomes integrated with the original design, making it impossible to precisely determine which sub-circuits are associated with which LUTs and the specific configurations of each LUT.

E. Overhead Analysis

The above observations hold significant implications for subsequent iterations of the entire circuit redaction, particularly for coarser-grained redaction. A key finding from this experiment is that fine-grained redaction exhibits better performance in terms of lower area overhead. This implies that, while maintaining circuit functionality and security, designers can employ more detailed redaction strategies to achieve better resource utilization in practical deployment. For smaller-scale benchmark circuits (e.g., pcode with gate count ≤ 1500), our experiments observed that replacing 15 sub-circuits resulted in an area overhead of approximately 13%. Meanwhile, for certain circuit benchmarks (e.g., FIR_filter), after sub-circuit replacement, negative area overhead was observed. This phenomenon arises due to the impact of optimization on some logic cones following LUT replacements.

V. CONCLUSION

In this paper, we introduce an innovative Register Transfer (RT) level obfuscation solution, namely EvoLUTe+, and extensively investigate the impact of IP redaction techniques

ranging from fine-grained to coarse-grained on its effectiveness. Unlike traditional methods, EvoLUTe+ employs smaller reconfigurable components (LUTs) for unique and fine-grained redaction. In EvoLUTe+, we introduce a mechanism of isolation, splitting, coloring, scoring, and redaction, enabling designers to apply redaction at an appropriate granularity, thus striking a balance between flexibility and protection. Through validation, we demonstrate that IP redaction at an appropriate granularity can offer the same security resilience at a lower cost. We also validate the robustness of EvoLUTe+ under different attack scenarios, including SAT attacks, BMC attacks, and machine-learning-based attacks without access to an oracle. Experimental results further underscore that IP redaction at an appropriate granularity can provide the same security resilience at a reduced cost, offering a robust solution for circuit protection and obfuscation.

REFERENCES

- [1] B. Shakya, M. M. Tehranipoor, S. Bhunia, and D. Forte, "Introduction to hardware obfuscation: Motivation, methods and evaluation," *Hardware Protection through Obfuscation*, pp. 3–32, 2017.
- [2] M. Tehranipoor and C. Wang, *Introduction to hardware security and trust*. Springer Science & Business Media, 2011.
- [3] H. M. Kamali, K. Z. Azar, F. Farahmandi, and M. Tehranipoor, "Advances in logic locking: Past, present, and prospects," *Cryptology ePrint Archive*, 2022.
- [4] A. B. Kahng, J. Lach, W. H. Mangione-Smith, S. Mantik, I. L. Markov, M. Potkonjak, P. Tucker, H. Wang, and G. Wolfe, "Constraint-based watermarking techniques for design ip protection," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 10, pp. 1236–1252, 2001.
- [5] Y. Alkabani, F. Koushanfar *et al.*, "Active hardware metering for intellectual property protection and security," in *USENIX security symposium*, vol. 20, 2007, pp. 1–20.
- [6] J. Rajendran, M. Sam, O. Sinanoglu, and R. Karri, "Security analysis of integrated circuit camouflaging," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 709–720.
- [7] J. Lee, M. Tehranipoor, C. Patel, and J. Plusquellic, "Securing scan design using lock and key technique," in *20th IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems (DFT'05)*. IEEE, 2005, pp. 51–62.
- [8] M. S. Rahman, R. Guo, H. M. Kamali, F. Rahman, F. Farahmandi, M. Abdel-Moneum, and M. Tehranipoor, "O'clock: lock the clock via clock-gating for soc ip protection," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, 2022, pp. 775–780.
- [9] J. A. Roy, F. Koushanfar, and I. L. Markov, "Epic: Ending piracy of integrated circuits," in *Proceedings of the conference on Design, automation and test in Europe*, 2008, pp. 1069–1074.
- [10] J. Rajendran, Y. Pino, O. Sinanoglu, and R. Karri, "Security analysis of logic obfuscation," in *Proceedings of the 49th annual design automation conference*, 2012, pp. 83–89.

- [11] M. Yasin, B. Mazumdar, J. J. Rajendran, and O. Sinanoglu, "Sarlock: Sat attack resistant logic locking," in *2016 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2016, pp. 236–241.
- [12] Y. Xie and A. Srivastava, "Mitigating sat attack on logic locking," in *Cryptographic Hardware and Embedded Systems—CHES 2016: 18th International Conference, Santa Barbara, CA, USA, August 17–19, 2016, Proceedings 18*. Springer, 2016, pp. 127–146.
- [13] P. Mohan, O. Atli, J. Sweeney, O. Kibar, L. Pileggi, and K. Mai, "Hardware redaction via designer-directed fine-grained efpga insertion," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1186–1191.
- [14] J. Bhandari, A. K. T. Moosa, B. Tan, C. Pilato, G. Gore, X. Tang, S. Temple, P.-E. Gaillardon, and R. Karri, "Exploring efpga-based redaction for ip protection," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [15] —, "Not all fabrics are created equal: Exploring efpga parameters for ip redaction," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.
- [16] B. Hu, J. Tian, M. Shihab, G. R. Reddy, W. Swartz, Y. Makris, B. C. Schaefer, and C. Sechen, "Functional obfuscation of hardware accelerators through selective partial design extraction onto an embedded fpga," in *Proceedings of the 2019 on Great Lakes Symposium on VLSI*, 2019, pp. 171–176.
- [17] X. Tang, E. Giacomin, A. Alacchi, B. Chauviere, and P.-E. Gaillardon, "Openfpga: An opensource framework enabling rapid prototyping of customizable fpgas," in *2019 29th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2019, pp. 367–374.
- [18] D. Koch, N. Dao, B. Healy, J. Yu, and A. Attwood, "Fabulous: An embedded fpga framework," in *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2021, pp. 45–56.
- [19] P. Beerel, M. Georgiou, B. Hamlin, A. J. Malozemoff, and P. Nuzzo, "Towards a formal treatment of logic locking," *Cryptology ePrint Archive*, 2022.
- [20] J. Zhou and X. Zhang, "Generalized sat-attack-resistant logic locking," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2581–2592, 2021.
- [21] H. M. Kamali, K. Z. Azar, H. Homayoun, and A. Sasan, "Full-lock: Hard distributions of sat instances for obfuscating circuits using fully configurable logic and routing blocks," in *Proceedings of the 56th Annual Design Automation Conference 2019*, 2019, pp. 1–6.
- [22] —, "Interlock: An intercorrelated logic and routing locking," in *Proceedings of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.
- [23] H. M. Kamali, K. Z. Azar, K. Gaj, H. Homayoun, and A. Sasan, "Lut-lock: A novel lut-based logic obfuscation for fpga-bitstream and asic-hardware protection," in *2018 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. IEEE, 2018, pp. 405–410.
- [24] S. Roshanisefat, H. M. Kamali, K. Z. Azar, S. M. P. Dinakararao, N. Karimi, H. Homayoun, and A. Sasan, "Dfssd: Deep faults and shallow state duality, a provably strong obfuscation solution for circuits with restricted access to scan chain," in *2020 IEEE 38th VLSI Test Symposium (VTS)*. IEEE, 2020, pp. 1–6.
- [25] R. S. Chakraborty and S. Bhunia, "Harpoon: An obfuscation-based soc design methodology for hardware protection," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 28, no. 10, pp. 1493–1502, 2009.
- [26] M. Fyrbiak, S. Wallat, J. Déchelotte, N. Albartus, S. Böcker, R. Tessier, and C. Paar, "On the difficulty of fsm-based hardware obfuscation," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 293–330, 2018.
- [27] A. R. Desai, M. S. Hsiao, C. Wang, L. Nazhandali, and S. Hall, "Interlocking obfuscation for anti-tamper hardware," in *Proceedings of the eighth annual cyber security and information intelligence research workshop*, 2013, pp. 1–4.
- [28] J. Dofe and Q. Yu, "Novel dynamic state-deflection method for gate-level design obfuscation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 2, pp. 273–285, 2017.
- [29] F. Koushanfar, "Provably secure active ic metering techniques for piracy avoidance and digital rights management," *IEEE Transactions on Information Forensics and Security*, vol. 7, no. 1, pp. 51–63, 2011.
- [30] M. Rostami, F. Koushanfar, and R. Karri, "A primer on hardware security: Models, methods, and metrics," *Proceedings of the IEEE*, vol. 102, no. 8, pp. 1283–1295, 2014.
- [31] A. Sehgal, V. Perelman, S. Kuryla, and J. Schonwalder, "Management of resource constrained devices in the internet of things," *IEEE Communications Magazine*, vol. 50, no. 12, pp. 144–149, 2012.
- [32] A. Musaddiq, Y. B. Zikria, O. Hahm, H. Yu, A. K. Bashir, and S. W. Kim, "A survey on resource management in iot operating systems," *IEEE Access*, vol. 6, pp. 8459–8482, 2018.
- [33] P. Subramanyan, S. Ray, and S. Malik, "Evaluating the security of logic encryption algorithms," in *2015 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 2015, pp. 137–143.
- [34] A. Alaql, D. Forte, and S. Bhunia, "Sweep to the secret: A constant propagation attack on logic locking," in *2019 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*. IEEE, 2019, pp. 1–6.
- [35] M. S. Rahman, A. Nahiyan, F. Rahman, S. Fazzari, K. Plaks, F. Farahmandi, D. Forte, and M. Tehranipoor, "Security assessment of dynamically obfuscated scan chain against oracle-guided attacks," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 4, pp. 1–27, 2021.
- [36] F. Corno, M. S. Reorda, and G. Squillero, "Rt-level itc'99 benchmarks and first atpg results," *IEEE Design & Test of computers*, vol. 17, no. 3, pp. 44–53, 2000.
- [37] T. R. Jensen and B. Toft, *Graph coloring problems*. John Wiley & Sons, 2011.
- [38] L. Amarú, P.-E. Gaillardon, and G. De Micheli, "The epfl combinational benchmark suite," in *Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS)*, no. CONF, 2015.
- [39] M. L. Laboratory, "Common evaluation platform," <https://github.com/mit-ll/CEP>, 2021.
- [40] R. Cavada, A. Cimatti, M. Dorigatti, A. Griggio, A. Mariotti, A. Micheli, S. Mover, M. Roveri, and S. Tonetta, "The nuxmv symbolic model checker," in *Computer Aided Verification: 26th International Conference, CAV 2014, Held as Part of the Vienna Summer of Logic, VSL 2014, Vienna, Austria, July 18–22, 2014. Proceedings 26*. Springer, 2014, pp. 334–342.
- [41] R. Brayton and A. Mishchenko, "Abc: An academic industrial-strength verification tool," in *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15–19, 2010. Proceedings 22*. Springer, 2010, pp. 24–40.
- [42] S. Roshanisefat *et al.*, "RANE: An Open-Source Formal De-obfuscation Attack for Reverse Engineering of Logic Encrypted Circuits," in *Great Lakes Symposium on VLSI (GLSVLSI)*, 2021, pp. 221–228.